

מחרוזות

string
string
STRING
string
string
string

ביחידה זו נלמד:

- מהי מחרוזת ?

- פעולות של מחרוזות

- דוגמאות מורכבות



- Length
- פניה לתו באמצעות []
- שרשור (+)
- השוואת מחרוזות: Equals ו- ==
- השוואת מחרוזות CompareTo
- חיפוש תווים.IndexOf
- חיפוש תת-מחרוזת.IndexOf
- חיפוש תו/תת-מחרוזת אחרון המחרוזת.LastIndexOf
- תת-מחרוזת.Substring
- החלפת חלק במחרוזת.Replace
- הוספת לאמצע המחרוזת.Insert
- מחיקה ממחרוזת.Remove
- שינוי רישיות ToUpper ו-ToLower

מהי מחרוזת ?

- מחרוזת היא טיפוס נתונים המכיל רצף של תווים, המשמשת בדרך כלל לביטוי מילים או משפטים.
- קבוע מסוג מחרוזת יש לעטוף בגרשיים.
- מיקום כל תו במחרוזת, ממוספר החל מ-0 ועד לסוף המחרוזת.
- ניתן לפנות לכל תו במחרוזת, בהתאם למיקומו באמצעות סוגריים מרובעים [].
- לא ניתן לשנות ישירות תו במחרוזת.

```
static void Main(string[] args)
{
    string s1 = "Hello World!";
    string s2 = "H";
    string s3 = "";

    Console.WriteLine ("s1 = |" + s1 + "|");
    Console.WriteLine ("s2 = |" + s2 + "|");
    Console.WriteLine ("s3 = |" + s3 + "|");
}
```

0	1	2	3	4	5	6	7	8	9	10	11
H	e	l	l	o		W	o	r	l	d	!

```
s1 = |Hello World!|
s2 = |H|
s3 = ||
```

מחרוזת ריקה עם 0 תווים.

גם רווח וסימן נחשבים תווים במחרוזת.

מחרוזת היא עצם (אובייקט)

- מחרוזת היא טיפוס מיוחד אבל מתנהגת כטיפוס רגיל בהצהרה והשמה.
- מחרוזת היא עצם (אובייקט), נלמד על זה בהמשך.
- משתנה מסוג מחרוזת היא הפניה לאובייקט.
- מחרוזת ריקה שונה ממחרוזת שאינה מאותחלת.

```
static void Main(string[] args)
{
    string s1 = "Hello World!";
    string s2 = "";
    string s3;

    Console.WriteLine ("s1 = |" + s1 + "|");
    Console.WriteLine ("s2 = |" + s2 + "|");
    Console.WriteLine ("s3 = |" + s3 + "|");
}
```

string: s1		→ "Hello World!"
string: s2		→ ""
string: s3	null	

```
s1 = |Hello World!|
s2 = ||
```

יש שימוש במשתנה שלא אותחל
- הקומפילר מציג שגיאה .

מחרוזת היא עצם (אובייקט) immutable

- מחרוזת היא טיפוס שברגע שנוצר ממנו עצם לא ניתן לשנות את ערכיו – immutable.
- כל ניסיון לשנות ישירות את ערך המחרוזת יגרור שגיאה.
- כל שימוש בפעולות המשנות את המחרוזת יגרור יצירת מחרוזת חדשה.

השמה בין מחרוזות

- השמה בין משתנים מטיפוס מחרוזת משנה את ההפניה.

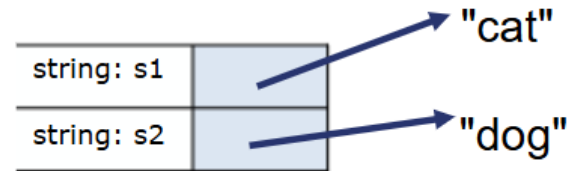
```
static void Main(string[] args)
{
    string s1 = "lion";
    string s2 = "dog";

    Console.WriteLine ("s1 = |" + s1 + "|");
    Console.WriteLine ("s2 = |" + s2 + "|");

    s1 = s2;
    Console.WriteLine ("s1 = |" + s1 + "|");
    Console.WriteLine ("s2 = |" + s2 + "|");

    s1 = "cat";

}
```



```
s1 = |lion|
s2 = |dog|

s1 = |dog|
s2 = |dog|
```

אין אפשרות לשנות את ערך המחרוזת.
כל שינוי מייצר מחרוזת חדשה.

השמה בין מחרוזות

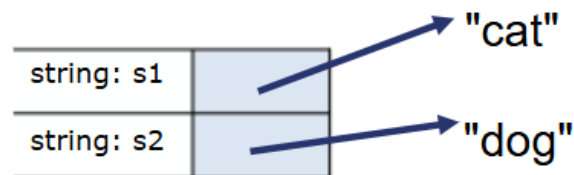
- השמה בין משתנים מטיפוס מחרוזת משנה את ההפניה.

```
static void Main(string[] args)
{
    string s1 = "lion";
    string s2 = "dog";

    Console.WriteLine ("s1 = |" + s1 + "|");
    Console.WriteLine ("s2 = |" + s2 + "|");

    s1 = s2;
    Console.WriteLine ("s1 = |" + s1 + "|");
    Console.WriteLine ("s2 = |" + s2 + "|");

    s1 = "cat";
    Console.WriteLine ("s1 = |" + s1 + "|");
    Console.WriteLine ("s2 = |" + s2 + "|");
}
```



```
s1 = |lion|
s2 = |dog|

s1 = |dog|
s2 = |dog|

s1 = |cat|
s2 = |dog|
```

אין אפשרות לשנות את ערך המחרוזת.
כל שינוי מייצר מחרוזת חדשה.

פעולת Console.ReadLine()

- מחזירה את מחרוזת הקלט מהמשתמש.

```
static void Main(string[] args)
{
    string s;

    Console.Write ("Enter the string: ");
    s = Console.ReadLine();
    Console.WriteLine ("You entered |" + s + "|");

    Console.Write ("Enter an Integer number: ");
    s = Console.ReadLine();
    int num1 = int.Parse (s);
    Console.WriteLine ("You entered |" + s + "| = " + num1 );

    Console.Write ("Enter a Double number: ");
    double num2 = double.Parse (Console.ReadLine());
    Console.WriteLine ("You entered the number : " + num2 );
}
```

```
Enter the string: Good Morning
You entered |Good Morning|
```

```
Enter the string:
You entered |
```

```
Enter an Integer number: 123
You entered |123| = 123
```

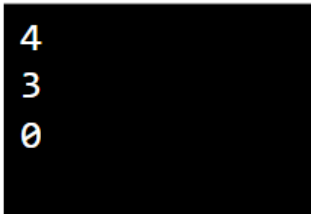
```
Enter a Double number: 5.67
You entered the number : 5.67
```


אורך מחרוזת : Length

- מחזירה את מספר התווים במחרוזת.
- נשים לב שזוהי אינה פעולה ולכן לא נפעיל אותה עם ().

```
static void Main(string[] args)
{
    string s1 = "lion";
    string s2 = "dog";
    string s3 = "";
    string s4;

    Console.WriteLine (s1.Length);
    Console.WriteLine (s2.Length);
    Console.WriteLine (s3.Length);
    Console.WriteLine (s4.Length);
}
```



4
3
0

שגיאה מאחר ו-s4 לא אותחל,
לא ניתן להפעיל עליו פעולות.

פניה לתו במחרוזת : []

- מחרוזת היא אוסף תווים, וניתן לפנות לתו ספציפי עפ"י מיקומו באמצעות [].
- מיקום התו הראשון הוא 0.
- נשתמש ב-Length כדי לקבל את אורך המחרוזת.

```
static void Main(string[] args)
{
    string s1 = "lion";

    Console.WriteLine (s1 + " starts with " + s1[0]);
    Console.WriteLine (s1 + " ends with " + s1[s1.Length - 1]);
    Console.WriteLine (s1[1]);
    Console.WriteLine (s1[2]);
    Console.WriteLine (s1[3]);
    Console.WriteLine (s1[4]);
}
```

```
lion starts with l
lion ends with n
i
o
n
```

```
Unhandled Exception: System.IndexOutOfRangeException:
Index was outside the bounds of the array.
at System.string.get_Chars(Int32 index)
at string.Program.Main(string[] args) in C:\Program.cs:line 20
```

מעבר על מחרוזת

- ניתן לעבור על האינדקסים של תווי המחרוזת בלולאה, מ-0 ועד אורכה.
- נמצא את מספר הרווחים במחרוזת כלומר את מספר המילים.

```
// return num of space character in string
static int CountWords(string str)
{
    if (str == "") return 0;

    int count = 0;
    for (int i=0; i < str.Length; i++)
        if (str[i] == ' ') count ++;
    return count+1;
}
```

```
static void Main(string[] args)
{
    string s = "i love u";
    Console.WriteLine(CountWords(s));
}
```

מעבר על מחרוזת

- ניתן לעבור על האינדקסים של תווי המחרוזת בלולאה.
- נבדוק האם מחרוזת היא פלינדרום (ניתן לקרוא אותה משני הכיוונים: level, hannah).
- נבדוק את התווים משני צידי המחרוזת - ראשון מול אחרון, שני מול לפני אחרון וכך הלאה.

```
static bool IsPalindrome(string str)
{
    int begin = 0, end = str.Length-1;
    while (begin < end)
    {
        if (str[begin] != str[end])
            return false;
        begin++;
        end--;
    }
    return true;
}
```

```
Is level palindrome? true
Is hannah palindrome? true
Is levd1 palindrome? false
```

```
static void Main(string[] args)
{
    string s1 = "level";
    Console.WriteLine("Is " + s1 + " palindrome ? " + IsPalindrome(s1));
    string s2 = "hannah";
    Console.WriteLine("Is " + s2 + " palindrome ? " + IsPalindrome(s2));
    string s3 = "levd1";
    Console.WriteLine("Is " + s1 + " palindrome ? " + IsPalindrome(s3));
}
```

מעבר על מחרוזת

- ניתן לעבור על האינדקסים של תווי המחרוזת בלולאה.
- נבדוק האם מחרוזת היא פלינדרום (ניתן לקרוא אותה משני הכיוונים – level, hannah).
- נבדוק את התווים משני צידי המחרוזת - ראשון מול אחרון, שני מול לפני אחרון וכך הלאה.

```
static bool IsPalindrome(string str)
{
    bool isPalindrom = true;
    for (int i=0; i< str.Length && isPalindrom ; i++)
    {
        isPalindrom = (str[i] == str[str.Length-1-i]);
    }
    return isPalindrom;
}
```

ואפשר גם כך....

```
Is level palindrome? true
Is hannah palindrome? true
Is levd1 palindrome? false
```

```
static void Main(string[] args)
{
    string s1 = "level";
    Console.WriteLine("Is " + s1 + " palindrome ? " + IsPalindrome(s1));
    string s2 = "hannah";
    Console.WriteLine("Is " + s2 + " palindrome ? " + IsPalindrome(s2));
    string s3 = "levd1";
    Console.WriteLine("Is " + s1 + " palindrome ? " + IsPalindrome(s3));
}
```

שרשור מחרוזות : +

- פעולת השרשור מאחדת מחרוזות למחרוזת חדשה אחת.

```
static void AddStrings()  
{  
    string s1, s2, s3;  
  
    Console.Write ("Enter first string: ");  
    s1 = Console.ReadLine();  
  
    Console.Write ("Enter second string: ");  
    s2 = Console.ReadLine();  
  
    s3 = s1 + " " + s2;  
    Console.WriteLine (s3);  
  
    Console.WriteLine (s1 + " " + s2 );  
}
```

```
Enter first string: good  
Enter second string: morning  
good morning  
good morning
```

שרשור מחרוזת עם מספר

- פעולות השרשור מתבצעות משמאל לימין והתוצר של כל חיבור כזה הוא מחרוזת.

```
static void AddStringAndNumber()
{
    int age;

    Console.Write ("How old are you? ");
    age = int.Parse(Console.ReadLine());

    string s = "Your age is " + age + " years old";
    Console.WriteLine (s);

    s = "Next year you will be " + age + 1 + " years old";
    Console.WriteLine (s);
}
```

How old are you? 17
Your age is 17 years old
Next year you will be 171 years old

שרשור מחרוזת עם מספר

- פעולות השרשור מתבצעות משמאל לימין והתוצר של כל חיבור כזה הוא מחרוזת.
- כדי שחיבור המספרים יתבצע לפני שרשור המחרוזות יש לעטוף את פעולת החיבור בסוגריים.
- השימוש בסוגריים משנה את סדר הביצוע.

```
static void AddStringAndNumber()  
{  
    int age;  
  
    Console.Write ("How old are you? ");  
    age = int.Parse(Console.ReadLine());  
  
    string s = "Your age is " + age + " years old";  
    Console.WriteLine (s);  
  
    s = "Next year you will be " + (age + 1) + " years old";  
    Console.WriteLine (s);  
}
```

```
How old are you? 17  
Your age is 17 years old  
Next year you will be 18 years old
```


חשוב!

פעולת השרשור על מחרוזת אינה משנה את המחרוזת,
אלא מחזירה ערך של מחרוזת חדשה!

```
static void Main(string[] args)
{
    string str1 = "It's a bea";
    string str2 = "utiful day";
    string str3 = str1 + str2;

    Console.WriteLine ("str1 = " + str1);
    Console.WriteLine ("str2 = " + str2);
    Console.WriteLine ("str3 = " + str3);

    str1 += str2; // same as str1 = str1 + str2
    Console.WriteLine ("str1 = " + str1);
    Console.WriteLine ("str2 = " + str2);
}
```

```
str1 = It's a bea
str2 = utiful day
str3 = It's a beautiful day
```

```
str1 = It's a beautiful day
str2 = utiful day
```



השוואת מחרוזות : Equals והאופרטור ==

- הפקודה Equals בודקת אם שתי מחרוזות זהות, כולל רישיות.
- לחילופין, בשפת C#, ניתן להשתמש באופרטור ==.
- ניתן לשלול באמצעות שילוב הפעולה עם האופרטור !

או באמצעות האופרטור !=

```
static void CompareStrings()  
{
```

```
    string s1 = "hello";  
    string s2 = "hello";  
    string s3 = "Hello";
```

```
    Console.WriteLine (s1.Equals(s2));  
    Console.WriteLine (s1.Equals(s3));  
    Console.WriteLine (s2.Equals(s3));
```

True
False
False



```
    Console.WriteLine (s1 == s2);  
    Console.WriteLine (s1 == s3);  
    Console.WriteLine (s2 == s3);
```

```
    Console.WriteLine (!s1.Equals(s2));  
    Console.WriteLine (!s1.Equals(s3));  
    Console.WriteLine (!s2.Equals(s3));
```

False
True
True



```
    Console.WriteLine (s1 != s2);  
    Console.WriteLine (s1 != s3);  
    Console.WriteLine (s2 != s3);
```

```
}
```

יחס גודל בין מחרוזות

- הפעולה Equals מחזירה רק אם שתי מחרוזות זהות או לא.
 - היא אינה מחזירה אינדיקציה מי מהן קטנה או גדולה יותר.

- מחרוזת נחשבת לקטנה יותר ממחרוזת אחרת אם היא קטנה ממנה מבחינה לקסיקוגרפית.

- מחרוזת א' קטנה ממחרוזת ב' אם היא מופיעה לפניו במילון.
 - מחרוזת א' גדולה ממחרוזת ב' אם היא מופיעה אחריה במילון.
 - החלטה זו מבוססת על השוואת ערכי ה-Ascii של התווים.

hello < world
world > hello

- כל מחרוזת עם אות גדולה תהיה קטנה ממחרוזת המתחילה עם אות קטנה כי ערכי ה-Ascii של האותיות הגדולות קטן מערכי ה-Ascii של האותיות הקטנות.

Decimal	Char	Decimal	Char
64	@	96	`
65	A	97	a
66	B	98	b
67	C	99	c
68	D	100	d
69	E	101	e
70	F	102	f
71	G	103	g
72	H	104	h
73	I	105	i
74	J	106	j
75	K	107	k
76	L	108	l
77	M	109	m
78	N	110	n
79	O	111	o
80	P	112	p
81	Q	113	q
82	R	114	r
83	S	115	s
84	T	116	t
85	U	117	u
86	V	118	v
87	W	119	w
88	X	120	x
89	Y	121	y
90	Z	122	z
91	[	123	{
92	\	124	
93	]	125	}
94	^	126	~
95	_	127	

יחס גודל בין מחרוזות : CompareTo

- הפעולה מקבלת מחרוזת כפרמטר ומחזירה :
 - ✓ **מספר שלילי** (-1) אם המחרוזת המפעילה **קטנה** מהמחרוזת שהועברה כפרמטר.
 - ✓ **מספר חיובי** (1) אם המחרוזת המפעילה **גדולה** מהמחרוזת שהועברה כפרמטר.
 - ✓ **0** אם שתי המחרוזות **זהות**.
- ההשוואה המתבצעת היא מבחינה לקסיקוגרפית.

```
static void CompareToExample()
{
    string s1, s2;

    Console.WriteLine ("Enter 2 strings: ");
    s1 = Console.ReadLine();
    s2 = Console.ReadLine();

    Console.WriteLine ("CompareTo returns " + s1.CompareTo(s2));
}
```

```
Enter 2 strings:
hello
world
CompareTo returns -1
```

```
Enter 2 strings:
world
hello
CompareTo returns 1
```

```
Enter 2 strings:
hello
hello
CompareTo returns 0
```

דוגמאות : CompareTo

```
static void CompareToExample()
{
    string s1, s2;
    int res;
    bool fContinue = true;

    while (fContinue)
    {
        Console.WriteLine ("Enter 2 strings, or both '!' to exit: ");
        s1 = Console.ReadLine();
        s2 = Console.ReadLine();

        if (s1.CompareTo("!") == 0 && s2.CompareTo("!") == 0)
            fContinue = false;
        else
        {
            res = s1.CompareTo(s2);
            Console.WriteLine ("CompareTo returns " + res + "\n");
        }
    }
    Console.WriteLine ("Bye");
}
```

דוגמאות : CompareTo

Enter 2 strings, or both '!' to exit:

hello

world

CompareTo returns -1

Enter 2 strings, or both '!' to exit:

world

hello

CompareTo returns 1

Enter 2 strings, or both '!' to exit:

hello

hello

CompareTo returns 0

Enter 2 strings, or both '!' to exit:

zz

aaa

CompareTo returns 1

Enter 2 strings, or both '!' to exit:

cc

ccc

CompareTo returns -1

Enter 2 strings, or both '!' to exit:

!

!

Bye

חיפוש תו : IndexOf

- הפעולה מקבלת תו ומחזירה את האינדקס הראשון בו הוא מופיע במחרוזת.
 - המיקום הראשון במחרוזת הוא 0.
- במידה והתו אינו מופיע במחרוזת יוחזר -1.

```
static void IndexOfExample()
{
    int index = "abc".IndexOf('a');
    Console.WriteLine (index);

    Console.WriteLine ("abc".IndexOf('b'));
    Console.WriteLine ("abc".IndexOf('c'));
    Console.WriteLine ("abc".IndexOf('d'));

    Console.WriteLine ("aaa".IndexOf('a'));
}
```

0

1

2

-1

0

חיפוש תת-מחרוזת : IndexOf

- הפעולה מקבלת מחרוזת ומחזירה את האינדקס הראשון בו היא מופיעה כתת-מחרוזת במחרוזת שהפעילה את הפעולה.
 - המיקום הראשון במחרוזת הוא 0.
- במידה המחרוזת שהתקבלה אינה מופיעה כתת-מחרוזת במלואה יוחזר -1.

```
static void IndexOfStringExample()  
{  
    Console.WriteLine ("abcdabcd".IndexOf("cd"));  
    Console.WriteLine ("abcdabcd".IndexOf("cde"));  
}
```

0 1 2 3 4 5 6 7

Console.WriteLine ("abcdabcd".IndexOf("cd"));

2

Console.WriteLine ("abcdabcd".IndexOf("cde"));

-1

מציאת תו / תת-מחרוזת החל מאינדקס מסוים

```
static void IndexOfFromExample()  
{  
    Console.WriteLine ("abcdabcd".IndexOf("cd", 3));  
}
```

מציאת תו / תת-מחרוזת החל מאינדקס מסוים

```
static void IndexOfFromExample()  
{  
    Console.WriteLine ("    0 1 2 3 4 5 6 7  
                        abcd".IndexOf("cd", 3));  
}
```

מציאת תו / תת-מחרוזת החל מאינדקס מסוים

```
static void IndexOfFromExample()  
{  
    Console.WriteLine ("    0 1 2 3 4 5 6 7  
                        abcd".IndexOf("cd", 3));  
    Console.WriteLine ("abcdabcd".IndexOf('a', 3));  
}
```

6

מציאת תו / תת-מחרוזת החל מאינדקס מסוים

- הפעולה מקבלת תו או מחרוזת ומיקום עבור התחלת החיפוש ומחזירה את האינדקס הראשון בו הם מופיעים במחרוזת שהפעילה את הפעולה, החל מהמיקום שהתקבל כפרמטר.
 - המיקום הראשון במחרוזת הוא 0.
 - המיקום שיוחזר הוא מיקום התו / תת-המחרוזת מתחילת המחרוזת.
- במידה והתו / המחרוזת שהתקבלו אינם מופיעים יוחזר -1.

```
static void IndexOfFromExample()
{
    Console.WriteLine ("    0 1 2 3 4 5 6 7  
                        abcd".IndexOf("cd", 3));
    Console.WriteLine ("    abcd".IndexOf('a', 3));
}
```

6

4

IndexOf והעמסת פעולות

תזכורת:

- **העמסת פעולות** היא כאשר יש לנו מספר פעולות עם שם זהה **בתנאי** שהן נבדלות אחת מהשניה במספר ובטיפוסי הפרמטרים שהן מקבלות.
- ראינו שלפעולה IndexOf קיימות 4 גרסאות :
 1. מקבלת תו כפרמטר.
 2. מקבלת מחרוזת כפרמטר.
 3. מקבלת תו כפרמטר ומיקום להתחלת החיפוש.
 4. מקבלת מחרוזת כפרמטר ומיקום להתחלת החיפוש.

LastIndexOf

- פעולה זו מחזירה את האינדקס **האחרון** בו מופיע התו / תת-מחרוזת שהתקבלו כפרמטר במחרוזת שהפעילה את הפעולה.
 - המיקום הראשון במחרוזת הוא 0.
 - המיקום שיוחזר הוא מיקום התו / תת-המחרוזת מתחילת המחרוזת.
- במידה והתו / המחרוזת שהתקבלו אינם מופיעים כלל יוחזר -1.

```
static void LastIndexOfExample()  
{  
    Console.WriteLine ("01234567" "abcdabcd".LastIndexOf("cd"));  
    Console.WriteLine ("abcdabcd".LastIndexOf('a'));  
}
```

6

4

יצירת תת-מחרוזת : Substring

- הפעולה מקבלת מיקום (בטווח) ומחזירה את תת-המחרוזת המתחילה החל ממיקום זה ועד לסוף המחרוזת.
- המיקום הראשון במחרוזת הוא 0.

```
static void SubStringExample()  
{  
    string sub = "01234567891011good morning".Substring(5);  
    Console.WriteLine (sub);  
  
    Console.WriteLine ("01234567891011good morning".Substring(0));  
    Console.WriteLine ("good morning".Substring(15));  
}
```

morning
good morning

Unhandled Exception: System.ArgumentOutOfRangeException: startIndex cannot be larger than length of string.

במקרה של חריגה מאורך המחרוזת,
נקבל שגיאה בזמן ריצה.

יצירת תת-מחרוזת באורך מסוים : Substring

- מקבלת מיקום (בטווח) ואורך ומחזירה את תת-המחרוזת המתחילה החל ממיקום זה באורך המבוקש.
 - המיקום הראשון במחרוזת הוא 0.
- בכל מקרה של חריגה מטווח אינדקס המחרוזת, נקבל שגיאה בזמן ריצה :
 - חריגה באינדקס ההתחלתי.
 - חריגה באורך הניתן החל מהמיקום ההתחלתי.

```
static void SubStringExample()  
{  
    Console.WriteLine ("good morning".Substring(5, 3));  
    Console.WriteLine ("good morning".Substring(5, 10));  
}
```

במקרה של חריגה מאורך המחרוזת,
נקבל שגיאה בזמן ריצה.

```
mor  
Unhandled Exception: System.ArgumentOutOfRangeException: Index and length must  
refer to a location within the string.
```


החלפת תת-מחרוזת : Replace

- הפעולה מקבלת שני תווים, ומחליפה כל מופע של התו הראשון בתו השני.

- במידה והתו המבוקש לא נמצא – לא תתבצע החלפה וגם לא תהיה שגיאה.

```
static void ReplaceChar()  
{  
    string s = "good morning";  
    Console.WriteLine (s.Replace('o', 'O'));  
    Console.WriteLine (s);  
}
```

```
gOOd mOrning  
good morning
```

- הפעולה מקבלת שתי מחרוזות ומחליפה כל מופע של המחרוזת הראשונה בשניה.

```
static void ReplaceString()  
{  
    Console.WriteLine ("good morning".Replace("morning", "night"));  
    Console.WriteLine ("abcdabc".Replace("ab", "ABC"));  
}
```

```
good night  
ABCcdABcC
```

באמור:

המחרוזת המקורית לא משתנה
אלא מוחזרת מחרוזת חדשה.

הפעולה : Insert

- הפעולה מייצרת מחרוזת חדשה, המכניסה את המחרוזת שהתקבלה כפרמטר בתוך המחרוזת המזמנת. ההכנסה מתבצעת החל ממיקום מסוים המתקבל כפרמטר.
- המיקום הראשון במחרוזת הוא 0.
- במידה והמיקום חורג מגבולות המחרוזת תתקבל שגיאת זמן ריצה – Run Time Error.

```
static void InsertExample()
```

```
{
```

```
0 1 2 3 4 5 6 7 8 9 10
```

```
string str = "C# is nice";
```

```
Console.WriteLine (str.Insert(6, "very "));
```

```
C# is very nice
```

```
6
```

```
int pos = str.IndexOf("nice");
```

```
Console.WriteLine (str.Insert(pos, "very "));
```

```
C# is very nice
```

```
}
```

באמור:

המחרוזת המקורית לא משתנה
אלא מוחזרת מחרוזת חדשה.

הפעולה : Remove

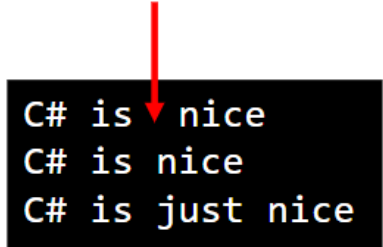
- הפעולה מייצרת מחרוזת חדשה.
- הפעולה מקבלת מיקום ומספר, ומסירה ממיקום זה את מספר התווים שהתקבל.
- המיקום הראשון במחרוזת הוא 0.

יש להקפיד שהמיקום והאורך בטווח, אחרת תהיה שגיאת זמן ריצה – Run Time Error.

```
static void RemoveExample()
{
    string str = "C# is very nice";
    int pos = str.IndexOf("very");
    Console.WriteLine (str.Remove(pos, "very".Length));
    Console.WriteLine (str.Remove(pos, "very".Length+1));

    string temp = str.Remove(pos, "very".Length + 1);
    6 pos = temp.IndexOf("nice");
    Console.WriteLine (temp.Insert(pos, "just "));
}
```

יש פה שני רווחים



C# is nice
C# is nice
C# is just nice

באמור:

המחרוזת המקורית לא משתנה
אלא מוחזרת מחרוזת חדשה.

שינוי רישיות : ToUpper ו-ToLower

- ToLower מחזירה מחרוזת חדשה כך שכל אות גדולה הופכת לקטנה.
- ToUpper מחזירה מחרוזת חדשה כך שכל אות קטנה הופכת לגדולה.

```
static void ToLowerUpper()  
{  
    Console.WriteLine("abc!12ABC#".ToLower());  
    Console.WriteLine("abc!12ABC#".ToUpper());  
}
```

abc!12abc#
ABC!12ABC#

באמור:

המחרוזת המקורית לא משתנה
אלא מוחזרת מחרוזת חדשה.

פיצול מחרוזת : Split (העשרה)

- הפעולה מקבלת תו מפריד ומחזירה מערך של תתי-מחרוזות מהמחרוזת המקורית אשר פוצלו ע"י התו המפריד.

```
// return num of space character in string
static int CountWords(string str)
{
    string[] ary = str.Split(' ');
    return ary.Length;
}
```

```
static void Main(string[] args)
{
    string s = "i love u";
    Console.WriteLine(CountWords(s));
}
```

פיצול מחרוזת : Split (העשרה)

- הפעולה Split יכולה לקבל כפרמטר מערך של תווים מפרידים. וכן ניתן להוסיף פרמטר נוסף כדי להתעלם ממחרוזות באורך 0.

```
// return num of space character in string
static int CountWords (string str)
{
    char[] chars = {' ', ',', '.', ' '};
    string[] ary = str.Split(chars, StringSplitOptions.RemoveEmptyEntries);
    return ary.Length;
}
```

```
static void Main(string[] args)
{
    string s = "i love C#,Java, Asp.net";
    Console.WriteLine(CountWords(s));
}
```

שאלת סיכום 1 : חיתוך מחרוזות – כל התווים המשותפים לשתי מחרוזות

כתבו פעולה המקבלת שתי מחרוזות.
הפעולה תחזיר מחרוזת חדשה ובה כל התווים שנמצאים בשתי המחרוזות.

לדוגמה:

happiness	מחרוזת 1
inspiration	מחרוזת 2
apins	מחרוזת תוצאה

אלגוריתם לפיתרון

1. נאתחל את מחרוזת התוצאה למחרוזת ריקה.
2. נעבור בלולאה על כל תווי המחרוזת הראשונה.
3. לכל תו במחרוזת הראשונה,
אם הוא נמצא במחרוזת הראשונה, וגם לא נמצא במחרוזת התוצאה – נצרף למחרוזת התוצאה.
4. החזרת מחרוזת התוצאה.

שאלת סיכום 1 : חיתוך מחרוזות – כל התווים המשותפים לשתי מחרוזות

// הפעולה מקבלת שתי מחרוזות
// הפעולה מחזירה את מחרוזת החיתוך - התווים המשותפים

```
static string Intersection(string str1, string str2)
{
    char ch;
    string intersectionStr = "";

    for (int i = 0; i < str1.Length; i++)
    {
        ch = str1[i];
        if (str2.IndexOf(ch) > -1 && intersectionStr.IndexOf(ch) == -1)
        {
            intersectionStr += str1[i];
        }
    }

    return intersectionStr;
}
```

לולאה עד לאורך המחרוזת.

התו במיקום i

צובר של התווים המשותפים

פעולה המחזירה את מיקום התו במחרוזת.
אם התו אינו במחרוזת, הפעולה תחזיר -1

שאלת סיכום 1 : חיתוך מחרוזות – כל התווים המשותפים לשתי מחרוזות

```
static void Main(string[] args)
{
    string str1 = "friendship";
    string str2 = "wonderful";

    Console.WriteLine (Intersection(str1,str2));
}
```

frend

שאלת סיכום 2 : אורך הרצף הארוך ביותר

כתבו פעולה המקבלת מחרוזת.
הפעולה תחזיר את אורכו של רצף התווים הזהים הארוך ביותר.

לדוגמה:

4 -	LLOOOVVVE	מחרוזת
1 -	inspiration	מחרוזת
0 -	""	מחרוזת

אלגוריתם לפיתרון

1. אם המחרוזת ריקה, נחזיר 0.
2. נאתחל מונה ל-1, ומקסימום ל-1.
3. נעבור בלולאה על תווי המחרוזת, מהתו השני.
 - 3.1 אם התו שווה לתו הקודם במחרוזת,
 - 3.1.1 נקדם את המונה.
 - 3.1.2 אם המונה גדול מהמקסימום, נעדכן את המקסימום.
 - 3.2 אחרת
 - 3.2.1 נאתחל את המונה ל-1.

שאלת סיכום 2 : אורך הרצף הארוך ביותר

```
static int LongestRepeatingLength(string input)
{
    if (input == "")
        return 0;

    int maxRepeatingLength = 1;
    int currentRepeatingLength = 1;

    for (int i = 1; i < input.Length; i++)
    {
        if (input[i] == input[i-1])
        {
            currentRepeatingLength++;
            if (currentRepeatingLength > maxRepeatingLength)
                maxRepeatingLength = currentRepeatingLength;
        }
        else
        {
            currentRepeatingLength = 1;
        }
    }

    return maxRepeatingLength;
}
```

// הפעולה מקבלת מחרוזת
// הפעולה מחזירה את אורך הרצף הארוך ביותר

מתחילים מ-1 כי בודקים כל תו עם התו הקודם.

הגדלת מונה ובדיקת מקסימום.

רצף חדש

שאלת סיכום 2 : אורך הרצף הארוך ביותר

```
static void Main(string[] args)
{
    string str = "rrrrreeemmmembbbbbber";

    Console.WriteLine (LongestRepeatingLength(str));
}
```

9

שאלת סיכום 3 : מיון מילים במחרוזת

כתבו פעולה המקבלת משפט ובו מספר מילים.
הפעולה תחזיר מחרוזת חדשה ובה המילים כשהן ממוינות בסדר אלפביתי.

לדוגמה:

elegantly dances forward a big gracefully cat
a big cat dances elegantly forward gracefully

משפט קלט
משפט פלט

פירוק הבעיה לתת - משימות

1. פירוק המשפט למילים.
2. בניית מחרוזת תוצאה .
3. נכניס כל מילה למקומה בתוך מחרוזת התוצאה.
4. החזרת מחרוזת התוצאה .

שאלת סיכום 3 : מיון מילים במחרוזת

כתבו פעולה המקבלת משפט ובו מספר מילים.
הפעולה תחזיר מחרוזת חדשה ובה המילים כשהן ממוינות בסדר אלפביתי.

לדוגמה :

משפט קלט

משפט פלט

elegantly dances forward a big gracefully cat
a big cat dances elegantly forward gracefully

פירוק הבעיה לתת - משימות

1. פירוק המשפט למילים.

2. בניית מחרוזת תוצאה .

3. נכניס כל מילה למקומה בתוך מחרוזת התוצאה.

4. החזרת מחרוזת התוצאה .

משימה פשוטה יחסית, מילים מופרדות ע"י רווח.
כיוון שלא ידוע מספר המילים, נשתמש בלולאת while
ונחפש את הרווח הבא, עד שלא יהיה כזה.

בכל פעם "נוריד" את המילה והרווח שאחריה מהמשפט,
וכך נגיע בסוף למילה האחרונה אחרי שעברנו על כל המילים.

שאלת סיכום 3 : מיון מילים במחרוזת

כתבו פעולה המקבלת משפט ובו מספר מילים.
הפעולה תחזיר מחרוזת חדשה ובה המילים כשהן ממוינות בסדר אלפביתי.

לדוגמה :

elegantly dances forward a big gracefully cat
a big cat dances elegantly forward gracefully

משפט קלט
משפט פלט

פירוק הבעיה לתת - משימות

נאתחל את מחרוזת התוצאה למחרוזת ריקה.

1. פירוק המשפט למילים.
2. בניית מחרוזת תוצאה .
3. נכניס כל מילה למקומה בתוך מחרוזת התוצאה.
4. החזרת מחרוזת התוצאה .

שאלת סיכום 3 : מיון מילים במחרוזת

כתבו פעולה המקבלת משפט ובו מספר מילים.
הפעולה תחזיר מחרוזת חדשה ובה המילים כשהן ממוינות בסדר אלפביתי.

לדוגמה :

elegantly dances forward a big gracefully cat
a big cat dances elegantly forward gracefully

משפט קלט
משפט פלט

פירוק הבעיה לתת - משימות

1. פירוק המשפט למילים.
2. בניית מחרוזת תוצאה .
3. נכניס כל מילה למקומה בתוך מחרוזת התוצאה.
4. החזרת מחרוזת התוצאה .

משימה מורכבת:

יש להשוות את המילה החדשה עם המילים במחרוזת,
ולהכניסה למקום הנכון.
לא נשכח את הרווח שיש להוסיף בין המילים.
מהו המקום הנכון ?

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

לדוגמה :

elegantly dances forward a big gracefully cat
a big cat dances elegantly forward gracefully

משפט קלט
משפט פלט

משפט קלט

משפט פלט

elegantly dances forward a big gracefully cat

elegantly

אם מחרוזת התוצאה ריקה,
המילה תתווסף כמילה ראשונה

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

לדוגמה :

elegantly dances forward a big gracefully cat
a big cat dances elegantly forward gracefully

משפט קלט
משפט פלט

משפט קלט

משפט פלט

elegantly dances forward a big gracefully cat
dances forward a big gracefully cat

elegantly
dances elegantly

אם המילה יותר "קטנה" מהמילה הראשונה,
היא תכנס לפנייה

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

לדוגמה :

elegantly dances forward a big gracefully cat
a big cat dances elegantly forward gracefully

משפט קלט
משפט פלט

משפט קלט

משפט פלט

elegantly dances forward a big gracefully cat
dances forward a big gracefully cat

elegantly
dances elegantly

אם המילה יותר "קטנה" מהמילה הראשונה,
היא תכנס לפנייה

"קטנה" = מופיעה לפני במילון

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

לדוגמה :

משפט קלט

משפט פלט

elegantly dances forward a big gracefully cat
a big cat dances elegantly forward gracefully

משפט קלט

elegantly dances forward a big gracefully cat
dances forward a big gracefully cat
forward a big gracefully cat

משפט פלט

elegantly
dances elegantly
dances elegantly forward

נעבור בלולאה על המילים במשפט.
כל עוד המילה יותר "גדולה" מהמילה
במשפט, נעבור למילה הבאה.

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

לדוגמה :

משפט קלט

משפט פלט

elegantly dances forward a big gracefully cat

a big cat dances elegantly forward gracefully

משפט קלט

משפט פלט

elegantly dances forward a big gracefully cat

dances forward a big gracefully cat

forward a big gracefully cat

elegantly

dances elegantly

dances elegantly forward

נעבור בלולאה על המילים במשפט.
כל עוד המילה יותר "גדולה" מהמילה
במשפט, נעבור למילה הבאה.

אם המשפט "נגמר", המילה תכנס
כמילה אחרונה במשפט.

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

לדוגמה :

משפט קלט

משפט פלט

elegantly dances forward a big gracefully cat

a big cat dances elegantly forward gracefully

משפט קלט

משפט פלט

elegantly dances forward a big gracefully cat

dances forward a big gracefully cat

forward a big gracefully cat

a big gracefully cat

big gracefully cat

elegantly

dances elegantly

dances elegantly forward

a dances elegantly forward

a big dances elegantly forward

נעבור בלולאה על המילים במשפט.

- כל עוד המילה יותר "קטנה" מהמילה במשפט, נעבור למילה הבאה,

- כשהמילה במשפט יותר גדולה, נכניס את המילה לפניה.

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

לדוגמה :

elegantly dances forward a big gracefully cat
a big cat dances elegantly forward gracefully

משפט קלט
משפט פלט

משפט קלט

משפט פלט

elegantly dances forward a big gracefully cat
dances forward a big gracefully cat
forward a big gracefully cat
a big gracefully cat
big gracefully cat
gracefully cat
cat

elegantly
dances elegantly
dances elegantly forward
a dances elegantly forward
a big dances elegantly forward
a big dances elegantly forward gracefully
a big cat dances elegantly forward gracefully

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

```
static string InsertInPlace(string sorted, string word) // הפעולה מקבלת מחרוזת שהיא משפט ממוין ומילה  
{ // מחזירה מחרוזת עם המילה, תוך שמירה על הסדר
```

```
    if (sorted.Length == 0)  
        return word;
```

אם המשפט ריק, נחזיר את המילה

```
    string start = "";  
    int spacePos = sorted.IndexOf(' ');
```

- נאתחל מחרוזת למחרוזת ריקה,
- שתפקידה לשמור את המילים ה"קטנות" יותר מהמילה.
- נעבור בלולאה על המילים של המשפט הממוין, וכל מילה יותר "קטנה", נעביר לסוף מחרוזת זו (מחרוזת ההתחלה).

```
}
```


משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

```
static string InsertInPlace(string sorted, string word)
{
    if (sorted.Length == 0)
        return word;

    string start = "";
    int spacePos = sorted.IndexOf(' ');

    while (spacePos >= 0)
    {
        // הפעולה מקבלת מחרוזת שהיא משפט ממוין ומילה
        // מחזירה מחרוזת עם המילה, תוך שמירה על הסדר

        // נחפש את תו הרווח הראשון במשפט
        // כל עוד יש תו רווח
    }
}
```

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

// הפעולה מקבלת מחרוזת שהיא משפט ממוין ומילה
// מחזירה מחרוזת עם המילה, תוך שמירה על הסדר

```
static string InsertInPlace(string sorted, string word)  
{
```

```
    if (sorted.Length == 0)  
        return word;
```

```
    string start = "";  
    int spacePos = sorted.IndexOf(' ');
```

```
    while (spacePos >= 0)  
    {
```

```
        if (word.CompareTo(sorted.Substring(0, spacePos)) < 0 )  
            return start + word + " " + sorted;
```

```
        start += sorted.Substring(0, spacePos + 1);  
        sorted = sorted.Substring(spacePos + 1);  
        spacePos = sorted.IndexOf(' ');
```

```
    }
```

האם המילה קטנה יותר ?

הפעולה מחזירה תת מחרוזת שהיא
המילה הראשונה במשפט -
מאינדקס 0 עד מיקום הרווח.

```
}
```

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

```
static string InsertInPlace(string sorted, string word)
{
    if (sorted.Length == 0)
        return word;

    string start = "";
    int spacePos = sorted.IndexOf(' ');

    while (spacePos >= 0)
    {
        if (word.CompareTo(sorted.Substring(0, spacePos)) < 0 )
            return start + word + " " + sorted;

        start += sorted.Substring(0, spacePos + 1);
        sorted = sorted.Substring(spacePos + 1);
        spacePos = sorted.IndexOf(' ');
    }
}
```

// הפעולה מקבלת מחרוזת שהיא משפט ממוין ומילה
// מחזירה מחרוזת עם המילה, תוך שמירה על הסדר

אם המילה יותר "קטנה", נחבר את :
- מחרוזת ההתחלה, את המילה ואת שאר המשפט,
- ונחזיר.

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

// הפעולה מקבלת מחרוזת שהיא משפט ממוין ומילה
// מחזירה מחרוזת עם המילה, תוך שמירה על הסדר

```
static string InsertInPlace(string sorted, string word)
{
    if (sorted.Length == 0)
        return word;

    string start = "";
    int spacePos = sorted.IndexOf(' ');

    while (spacePos >= 0)
    {
        if (word.CompareTo(sorted.Substring(0, spacePos)) < 0 )
            return start + word + " " + sorted;

        start += sorted.Substring(0, spacePos + 1);
        sorted = sorted.Substring(spacePos + 1);
        spacePos = sorted.IndexOf(' ');
    }
}
```

אם המילה שלנו יותר "גדולה",
- נעביר את המילה התורנית במשפט לסוף משפט - התוצאה,
- ונסיר אותה מהמשפט הממוין.

נחפש את המילה הבאה (הרווח הבא)

}

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

// הפעולה מקבלת מחרוזת שהיא משפט ממיון ומילה
// מחזירה מחרוזת עם המילה, תוך שמירה על הסדר

```
static string InsertInPlace(string sorted, string word)
{
    if (sorted.Length == 0)
        return word;

    string start = "";
    int spacePos = sorted.IndexOf(' ');

    while (spacePos >= 0)
    {
        if (word.CompareTo(sorted.Substring(0, spacePos)) < 0 )
            return start + word + " " + sorted;

        start += sorted.Substring(0, spacePos + 1);
        sorted = sorted.Substring(spacePos + 1);
        spacePos = sorted.IndexOf(' ');
    }

    if (word.CompareTo(sorted) < 0)
        return start + word + " " + sorted;

    return start + sorted + " " + word;
}
```

אין רווח – הסתיים המשפט :
- האם המילה קטנה מהמילה האחרונה ?
- אם כן, תכנס לפניה, אם לא, אחריה.

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

// הפעולה מקבלת מחרוזת שהיא משפט ממוין ומילה
// מחזירה מחרוזת עם המילה, תוך שמירה על הסדר

```
static string SortWords(string input)
{
```

```
    string sorted = "";
    string word;
```

נאתחל את מחרוזת התוצאה
(הממויינת) למחרוזת ריקה

```
    if (input.Length == 0)
        return sorted;
```

```
    int spacePos = input.IndexOf(' ');
```

```
    while (spacePos != -1)
    {
```

```
        word = input.Substring(0, spacePos);
        sorted = InsertInPlace(sorted, word);
        input = input.Substring(spacePos + 1);
        spacePos = input.IndexOf(' ');
```

```
    }
```

```
    sorted = InsertInPlace(sorted, input);
    return sorted;
```

```
}
```

טיפול במילה האחרונה

כמו קודם,
- נמצא את הרווח הראשון,
- נסיר את המילה,
- נשלח אותה לפעולה המכניסה מילה למשפט ממויין,
- ולא נשכח לדרוס את המשתנה
במחרוזת החוזרת מהפעולה

משימה מורכבת : הכנסת מילה למחרוזת מילים ממויינת

```
static void Main(string[] args)
{
    string str = "Remember, every day is a chance to learn something new and " +
        "exciting, just like an adventure waiting to be discovered";
    Console.WriteLine(SortWords(str));
}
```

```
a adventure an and be chance day discovered every exciting is just learn like new Remember something to to waiting
```

ביחידה זו למדנו:

- מהי מחרוזת ?

- פעולות של מחרוזות

- דוגמאות מורכבות



- Length

- פניה לתו באמצעות []

- שרשור (+)

- השוואת מחרוזות: Equals ו- =

- השוואת מחרוזות CompareTo

- חיפוש תווים IndexOf

- חיפוש תת-מחרוזת IndexOf

- חיפוש תו/תת-מחרוזת אחרון המחרוזת LastIndexOf

- תת-מחרוזת Substring

- החלפת חלק במחרוזת Replace

- הוספת לאמצע המחרוזת Insert

- מחיקה ממחרוזת Remove

- שינוי רישיות ToUpper ו- ToLower